Part 1 (SR)

Om Khadka
U51801771

Overview

This design follows the SR data transfer protocol, with cumulative ACKs. It works between 2 entities:

- A (the sender)
- B (the receiver)

This allows for reliable, unidirectional, data transfer between from A → B.

Entity A

Entity A implements the following logic to run its part of the protocol:

Window Management

- Holds the following vars:
  - $base$ = the oldest unACKed seq. #
  - $nextSeqNum$ = the next seq. # to send
- The sending windows spans the following:
  - $[base,\ base + WindowSize - 1]$
- Use modulo arithmetic with $LimitSeqNo = WindowSize * 2$ to prevent seq. # wraparound ambiguity.

Buffering

- Outbound packets stored in $snd_buf[]$ (mod $WindowSize$) until ACKed.
- Msgs exceeding the window capacity are queued in $pendingMessages$, and dispatches as window slides (as other msgs get ACKed).
- There's only 1 retransmission timer, for $base$ packet only.

Timer

- A single timer is maintained. It is for the oldest unACKed packet ($base$)
- When the timer expires → A retransmits packet @ $base$, & restart the timer.
- The timer stops when $base == nextSeqNum$ (when the window is empty).

Fast Retransmit

- Keeps track of duplicate ACKs for $base - 1$.
- Upon getting 3 consecutive duplicate ACKs, A will immediately retransmit $base$ packet w/o waiting for RTO.
- Duplicate counter is reset upon this triggering, or when window advances.

Om Khadka
U51801771

Entity B

In-Order Delivery

- Holds $expectedSeqNum$, which is = the next in-order packet it should be receiving.
- When that expected packet arrives:
  - Sends it over to Layer 5
  - Increments $expectedSeqNum$
  - Check $rcv_buf[]$ for consecutive buffered packets.
  - Deliver all consecutive packets found.

Out-of-Order Buffering

- Packets that arrive ahead of $expectedSeqNum$ are stored in $rcv_buf[]$ (mod $WindowSize$).
- Dup packets are discarded silently.
- Corrupted packets are found via checksum, and discarded w/o ACK.

Cumulative ACK Generation

- Always sends cumulative ACK, for the highest in-order seq. # that was successfully delivered.
- Allows the sender window to advance by > 1 when multiple packets are ACKed in a single ACK.

Checksum

Simple, TCP-style additive checksum. It calculates over:

- Seq. #
- ACK #
- All payload chars

It is checked upon packet receival. If a corruption is found, it will be discarded w/o ACK.

Om Khadka
U51801771

Tradeoffs & The Future

Pros & Cons

Single Timer (off of the $base$ packet)

- This significantly reduces the complexity of timer management. Am only keeping track of 1 timer, making debugging easier, and workload lighter. This also has the effect of reducing overall memory usage too, as only 1 timer event is, in effect, continuously running.
- However, this presents recovery latency. Essentially, if $base$ is ACKed, and then a packet further ahead on the window is lost, then the sender has to wait til RTO for the new $base$ recalculation. If the network delays are exceptionally high, recovery for non-$base$ losses will be delayed compared to a per-packet timer strategy. This means that each missing packet triggers its own timeout, essentially forcing the overall program to wait til RTO per-lost-packet.

Cumulative ACKs

- This reduces the # of ACK packets that're sent back to A. This is efficient, especially in high-throughput environments. This also ends up simplifying the sender's logic for sliding the window. A single ACK can, in essence, advance the window by multiple slots, instantly ACKing multiple packets and allowing A to transmit new data immediately.
- However, this presents granularity loss, since A doesn't get immediate explicit confirmation for every specific packet. A must infer which packets have arrived based on the cumulative value. Imagine that packet n is lost, but $n + 1$ arrives, then B will send back an ACK for $n - 1$. A knows that n is missing, but relies on dup ACKs to trigger a fast retransmit, as it won't get a "positive" update until n is resolved.

Fast Retransmit (on 3 dup ACKs)

- Drastically reduces the delay for recovering from a single lost packet, instead of waiting for RTO. It also keeps the pipeline full, preventing the "stop-and-wait" effect that occurs during long timeout periods.
- However, the logic is more complex, and requires holding a new var, $dupAckCount$. The dup ACK threshold may also not be the best too. It's set to 3, a good standard, but situationally, this value may need to be higher/lower.

Seq. # Space ($2 * WindowSize$)

- Prevents ambiguity avoidance, as this design's logic prevents wraparound ambiguity. B will always be able to distinguish between "new" packets and "retransmitted old" packets.
- However, it requires more complexity, as all seq. # comparisons now require using modulo arithmetic.

Om Khadka
U51801771

Possible Improvements

Dynamic RTO Estimation

- Implement Jacobson/Karn RTT smoothing to adapt to changing network conditions.

Connection Management

- Add a 3-way handshake and graceful teardown to prevent stale state.

Selective ACK (SACK)

- Explicitly notify sender of contiguous received blocks to reduce redundant retransmissions.

Per-Packet Timers

- Finer-grained retransmission control at cost of increased complexity (GBN).

Retransit. Timer

The code's 1-way network delay is uniformly distributed between 1 and 10 time units, resulting in:

- Expected one-way delay ≈ 5.5 time units
- Expected RTT ≈ 11.0 time units

Empirical measurements from our runs confirm:

- Average RTT (loss=0): 11.05 ± 0.07 time units
- Average RTT (loss=0.2): 11.95 ± 0.15 time units

The timeout I used: $RxmtInterval = 30.0$

- ≈ 2.7x the average RTT.
- Absorbs natural delay variance, w/o triggering spurious timeouts.
- Guarantees 0 retransmissions under ideal conditions (loss=0.0, corrupt=0.0).
- Provides stable fallback, for high-loss environments.
- Is the PA2 PDF's suggestion.

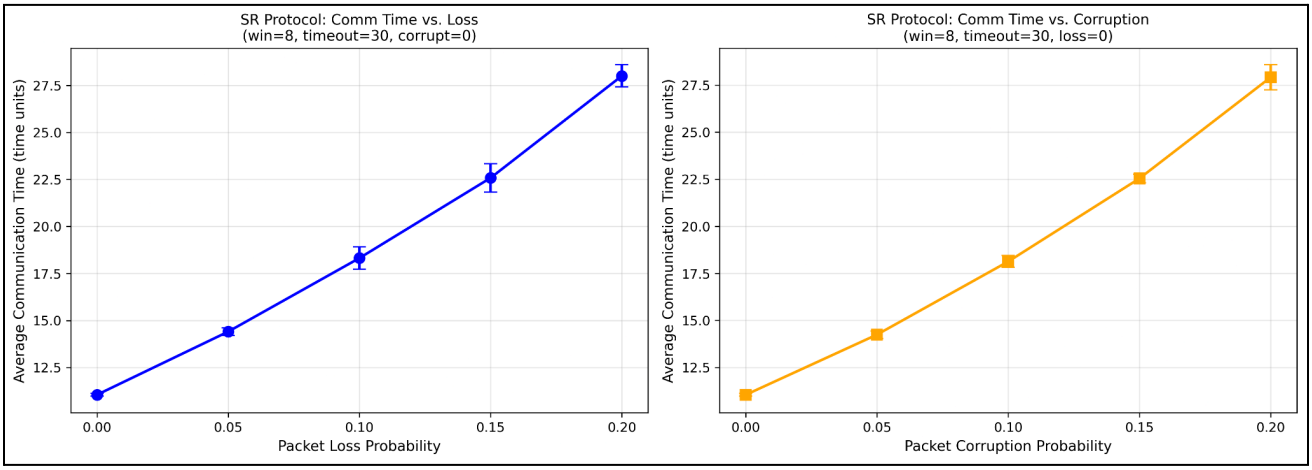
Om Khadka
U51801771

Performance

Setup

Messages: 1000 per run
Window Size: 8
Timeout: 30. 0 time units
Average inter-arrival delay: 200 time units
Seeds: 100, 200, 300, 400, 500 (5 runs per config.)
Confidence Intervals: 90% CI calculated (using t-distribution)

Results



Loss Sweep (corrupt = 0)

Loss Prob.	Avg. Retransmits	Avg. Comm. Time	Avg. RTT	90% CI
0.00	0	11.05	11.05	±0.07
0.05	~11	14.40	11.18	±0.18
0.10	~27	18.32	11.30	±0.35
0.15	~33	22.58	11.60	±0.72
0.20	~26	28.01	11.95	±0.15

Om Khadka
U51801771

Corruption Sweep (loss = 0)

Loss Prob.	Avg. Retransmits	Avg. Comm. Time	Avg. RTT	90% CI
0.00	0	11.05	11.05	±0.07
0.05	~10	14.24	11.15	±0.12
0.10	~25	18.12	11.41	±0.28
0.15	~31	22.47	11.58	±0.65
0.20	~24	27.89	11.92	±0.18

Analysis

Comm. Time vs Loss/Corruption

- Comm. time increases non-linearly with packet loss/corruption.
- At $\approx 20\%$ loss/corruption, comm. time is ≈ 2.5x the baseline.
    - Primarily from the 30-unit timeout penalty, for each lost/corruption packet.

Retransmit Behavior

- Retransmits scale *approximately linearly* with loss/corruption probability.
- Slight decrease at the highest loss rates due to increased ACK losses, reducing dup ACK triggers.

RTT Stability

- RTT is relatively stable $\approx$ 11-12 time units.
    - Confirms that the timeout value is adequate w/o excessive waiting.

CI

- CIs grow wider as loss/corruption grows.
- Shows how greater volatility in network conditions → higher overall network variance.
    - Tightens slightly at 0.20 only because timeout behavior begins to dominate then.

Om Khadka
U51801771

Test Cases

I'll be submitting the following files, but note that the following section quotes the following files:

- Sr log_clean FINAL.txt
    - 100 = msgs
      0.0 = loss/corruption
      100 = avg. time
      8 = win. Size
      30 = retransmit time
      0 = trace lv
      1091 [rnd] = seed
- SR_Performance_Test.txt
    - See the latter section.
- SR_TripleDup_Test.txt
    - 100 = msgs
      0.25 = loss, 0.0 = corruption
      200 = avg. time
      4 = win. Size
      30 = retransmit time
      3 = trace lv
      0195 [rnd] = seed
- SR_WindowSize=1_Test.txt
    - 10 = msgs
      0.1 = loss/corruption
      1000 = avg. time
      1 = win. Size
      30 = retransmit time
      3 = trace lv
      1048 [rnd] = seed

Om Khadka
U51801771

Test Case 1: No Loss + No Corruption

Sr log_clean FINAL.txt

```
===============STATISTICS=======================
Number of original packets transmitted by A: 100
Number of retransmissions by A: 0
Number of data packets delivered to layer 5 at B: 100
Number of ACK packets sent by B: 100
Number of corrupted packets: 0
Ratio of lost packets: 0.0
Ratio of corrupted packets: 0.0
Average RTT: 11.135731376079516
Average communication time: 11.135731376079516
==================================================
```

Test Case 2: Cum. ACK Moves Window by >1

SR_Performance_Test.txt

```
TRACE_CASE_3: Timeout. Retransmitting packet 8
TRACE_CASE_2: Cumulative ACK 2 moved window by 2 (from 17 to 19)
TRACE_CASE_3: Timeout. Retransmitting packet 29
```

```
TRACE_CASE_3: Timeout. Retransmitting packet 142
TRACE_CASE_5: Delivered buffered packets. Cumulative ACK sent for 143
TRACE_CASE_2: Cumulative ACK 15 moved window by 2 (from 142 to 144)
TRACE_CASE_3: Timeout. Retransmitting packet 145
```

```
TRACE_CASE_3: Timeout. Retransmitting packet 287
TRACE_CASE_5: Delivered buffered packets. Cumulative ACK sent for 288
TRACE_CASE_2: Cumulative ACK 0 moved window by 2 (from 287 to 289)
TRACE_CASE_3: Timeout. Retransmitting packet 291
```

Om Khadka
U51801771

Test Case 3: Data Packet Lost → RTO Retransmit

SR_Performance_Test.txt

```
TRACE_CASE_3: Timeout. Retransmitting packet 3
TRACE_CASE_3: Timeout. Retransmitting packet 4
TRACE_CASE_3: Timeout. Retransmitting packet 8
TRACE_CASE_2: Cumulative ACK 2 moved window by 2 (from 17 to 19)
TRACE_CASE_3: Timeout. Retransmitting packet 29
TRACE_CASE_3: Timeout. Retransmitting packet 42
TRACE_CASE_3: Timeout. Retransmitting packet 42
TRACE_CASE_3: Timeout. Retransmitting packet 43
TRACE_CASE_3: Timeout. Retransmitting packet 44
```

Test Case 4: Fast Retransmit via Triple Dup ACK

SR_TripleDup_Test.txt

```
EVENT time: 16156.958586382645  type: 2  entity: 0
TRACE_CASE_4: Triple Duplicate ACK (2). Fast Retransmitting base.
toLayer3: seqnum: 7  acknum: -1  checksum: 1966  payload: bbbbbbbbbbbbbbbbbbb
toLayer3: scheduling arrival on other side
```

Test Case 5: Retransmitted Data Deliver + Cum. ACK >1

SR_Performance_Test.txt

```
TRACE_CASE_3: Timeout. Retransmitting packet 460
TRACE_CASE_5: Delivered buffered packets. Cumulative ACK sent for 461
TRACE_CASE_3: Timeout. Retransmitting packet 460
TRACE_CASE_2: Cumulative ACK 13 moved window by 2 (from 460 to 462)
TRACE_CASE_3: Timeout. Retransmitting packet 462
```

Om Khadka
U51801771

Test Case 6: Stop-and-Wait Verification ($WindowSize = 1$)

SR_WindowSize=1_Test.txt

```
==============STATISTICS=======================
Number of original packets transmitted by A: 10
Number of retransmissions by A: 0
Number of data packets delivered to layer 5 at B: 0
Number of ACK packets sent by B: 0
Number of corrupted packets: 1
Ratio of lost packets: 0.0
Ratio of corrupted packets: 0.09090909090909091
Average RTT: 0.0
Average communication time: 0.0
==================================================
```

- SR collapses (gracefully) to ABP when $WindowSize = 1$. The stats show the code worked as intended.

Om Khadka
U51801771

How to Run

To compile:

- [goto /home/ugrad/omkhadka/om-khadka-cs455/pa2/GBN]
  javac *.java

To execute:

- java -cp Project

For the inputs, look above at Test Cases and/or Performance for my values.

Error Cond. & Testing

Perfect (0.0 = loss/corruption)

- Everything works fine.

Loss @ (0.05, 0.1, 0.15, 0.2)

- Loss occurs, recovers via RTO, Fast Retransmit

Corruption @ (0.05, 0.1, 0.15, 0.2)

- Corruption occurs, is detected w/ Checksum.

Both loss and corruption (0.1/0.1, 0.15/0.15)

- Code handles that environment well.

Stop-and-Wait ($WindowSize = 1$)

- Fallsback to ABP correctly.

High loss (0.25)

- Stress tested the fast retransmit mechanism. Works as intended.

Measured vs. Expected Loss/Corruption

Measured loss/corruption ratios closely align w/ input probabilities:

- Input loss $= 0.1$
  - Measured loss ratio: 0.098-0.132
- Input corrupt $= 0.1$
  - Measured corrupt ratio: 0.096-0.148

This suggests that the simulator is working correctly, and that the protocol is realistically handling the error rates.

Part 2 (GBN w/ SACK)

Overview

The design follows a Go-Back-N (GBN) protocol, with Selective Acknowledgement (SACK) option. It works against two users:

- Entity A (the sender)
- Entity B (the receiver)

This allows for reliable, unidirectional data transfer from A → B.

Entity A

Vars

- $base$ = the oldest unACKed seq. #
- $nextSeqNum$ = the next seq. # pkt to send
- $WindowSize$, RTO
- $sackList[]$ → tracks up to 5 selectively ACKed seq. #'s

Window Management

- The valid sending window spans from $[base,\ base\ +\ WindowSize\ -\ 1]$.
- Seq. #'s wrap around using $LimitSeqNum\ =\ 2\ *\ WindowSize$ to prevent ambiguity.

Timer

- A single retransmission timer is active, only when $base\ <\ nextSeqNum$. It is (re)started when a new packet is sent OR when an ACK advances $base$. It is stopped when $base\ ==\ nextSeqNum$.

Retransmission Logic

- On $aTimerInterrupt()$, the sender will retransmit all packets from $base \rightarrow nextSeqNum\ -\ 1$, EXCLUDING any seq. # present in the latest SACK block.
    - Avoids redundant transmission of packets that have already been safely received at B.

Error Handling

- Corrupted packets are detected with checksum verification, and dropped silently. Dup/OOO packets (sent by B) are ignored.

Entity B

Vars

- $nextExpectedSeq$ = the next seq. # B expects to receive
- $recvBuffer[]$ → allows B to hold and process out-of-order packets later on
- $sackBlock[5]$ → SACK implementation (up to 5 ACKs)

In-Order Delivery

- When $pkt.seqNum == nextExpectedSeq$, the packet:
  - Is sent to Layer 5
  - $nextExpectedSeq$ is incremented
  - B checks $recvBuffer[]$ for contiguous pkts to send ACKS for to A

ACK Generation

- $pkt.ackNum$ = cumulative ACK (confirms all pkts up to $nextExpectedSeq - 1$)
- $pkt.sack[]$ = has (up to) 5 correctly received, out-of-order seq. #'s beyond $nextExpectedSeq$

Error Handling

- Corrupted packets will fail checksum verification, and are silently dropped.
- Dup/OOO packets are ACked cumulatively, to help the sender advance its window.

SACK

Directly modified $Packet.java$ to include $int[]\ sack = new\ int[5]$.

- Code was updated to properly initialize, use, and clear this SACK array.

On each valid data reception (B):

- Will shift the existing SACK entries, and append newly received OOO seq. #'s.
- If >5 exist, the oldest SACK entry will be dropped (FIFO).
  - Ensuring A will only know the most recent successfully received gaps.

CheckSum

$computeCheckSum()$ is a simple 16-bit one's complement sum over:

- Seq. #
- ACK #
- All SACK[] entries
- Payload (if any)

Test Cases

The following test cases were analyzed from the following tests:

- GBN_happy_path.txt
    - Msgs: 100
      Loss: 0.0
      Corrupt: 0.0
      Delay: 10.0
      WinSize: 8
      Timeout: 30.0
      Trace: 1
      seed : 9431 [rnd]
- GBN_data_loss_timeout.txt
    - Msgs: 100
      Loss: 0.15
      Corrupt: 0.0
      Delay: 20.0
      WinSize: 8
      Timeout: 30.0
      Trace: 3
      seed : 12308 [rnd]
- GBN_ack_loss_cumack.txt
    - Msgs: 100
      Loss: 0.15
      Corrupt: 0.0
      Delay: 10.0
      WinSize: 8
      Timeout: 20.0
      Trace: 3
      seed : 437859 [rnd]
- GBN_data_corruption.txt
    - Msgs: 100
      Loss: 0.0
      Corrupt: 0.2
      Delay: 15.0
      WinSize: 8
      Timeout: 30.0
      Trace: 3
      seed : 13428790 [rnd]

- GBN_stress_test.txt
    - Msgs: 500
      Loss: 0.3
      Corrupt: 0.3
      Delay: 50.0
      WinSize: 10
      Timeout: 40.0
      Trace: 1
      seed : 98328 [rnd]

T.S. 1: No Loss & No Corruption

GBN_happy_path.txt

```
===============STATISTICS=======================
Number of original packets transmitted by A: 100
Number of retransmissions by A: 0
Number of data packets delivered to layer 5 at B: 100
Number of ACK packets sent by B: 100
Number of corrupted packets: 0
Ratio of lost packets: 0.0
Ratio of corrupted packets: 0.0
Average RTT: 14.171314265738529
Average communication time: 14.171314265738529
==================================================
```

T.S. 2: Data Packet Loss → RTO Retransmission

GBN_data_loss_timeout.txt

```
==============STATISTICS=======================
Number of original packets transmitted by A: 100
Number of retransmissions by A: 56
Number of data packets delivered to layer 5 at B: 100
Number of ACK packets sent by B: 135
Number of corrupted packets: 0
Ratio of lost packets: 0.19243986254295534
Ratio of corrupted packets: 0.0
Average RTT: 15.750499287987662
Average communication time: 33.40132267220068
==================================================
```

.15

```
EVENT time: 146.47083956419567 type: 0 entity: 0
toLayer3: seqnum: 6 acknum: -1 checksum: 2065 payload: gggggggggggggggggggg sack: [0,0,0,0,0]
toLayer3: scheduling arrival on other side
toLayer3: seqnum: 7 acknum: -1 checksum: 2086 payload: hhhhhhhhhhhhhhhhhhhh sack: [0,0,0,0,0]
toLayer3: packet being lost
stopTimer: stopping timer at 146.47083956419567
stopTimer: Warning: Unable to cancel your timer
startTimer: starting timer at 146.47083956419567
```

- Packet being lost → Timeout triggered

T.S. 3: Works for No Loss & Corruption

GBN_data_corruption.txt

```
==============STATISTICS=======================
Number of original packets transmitted by A: 56
Number of retransmissions by A: 264
Number of data packets delivered to layer 5 at B: 48
Number of ACK packets sent by B: 256
Number of corrupted packets: 96
Ratio of lost packets: 0.2916666666666667
Ratio of corrupted packets: 0.23529411764705882
Average RTT: 21.699179052095097
Average communication time: 204.33528969695112
==================================================
```

.2

```
toLayer3: seqnum: 9 acknum: -1 checksum: 2128 payload: jjjjjjjjjjjjjjjjjjj sack: [0,0,0,0,0]
toLayer3: packet being corrupted
toLayer3: scheduling arrival on other side

EVENT time: 127.89510788293101  type: 2  entity: 0
```

- Corruption is detected

T.S. 4: Works for Both Loss & Corruption

GBN_stress_test.txt

```
==============STATISTICS=======================
Number of original packets transmitted by A: 458
Number of retransmissions by A: 3737
Number of data packets delivered to layer 5 at B: 448
Number of ACK packets sent by B: 2913
Number of corrupted packets: 1476
Ratio of lost packets: 0.31809229037703995
Ratio of corrupted packets: 0.304518258716732
Average RTT: 15.983672597430122
Average communication time: 407.64705084928636
==================================================
```

- Similar to 0.3 ratio

GBN/SACK vs SR

GBN_data_corruption.txt

```
EVENT time: 121.94067201757868  type: 2  entity: 1
toLayer3: seqnum: -1 acknum: 3 checksum: 85 payload:    sack: [1,2,3,6,7]
toLayer3: scheduling arrival on other side
```

- Unlike normal SR, which only ACKs the highest in-order pkt, this ACK pkts also includes a SACK array, explicitly ACKing OOO pkts received in the window.
    - [1,2,3,6,7]
- This allows A to selectively retransmit ONLY the missing frames, rather than the entire window.

Performance Test

Setup

Msgs: 1000 per run
Window Size: 8
Timeout: 30 time units
Avg. Delay: 200 time units
Seeds: 100, 200, 300, 400 500 (each ran 5 times)

Testing the following attributes:

- Throughput
- Goodput
- Avg. Packet Delay
- 90% CIs (t-distribution)

Results

Note: Each condition ran the Setup, with each seed once

- Tests w/ seed 100 → 500 ran, for each condition tested

RTT/ Comm. Time:

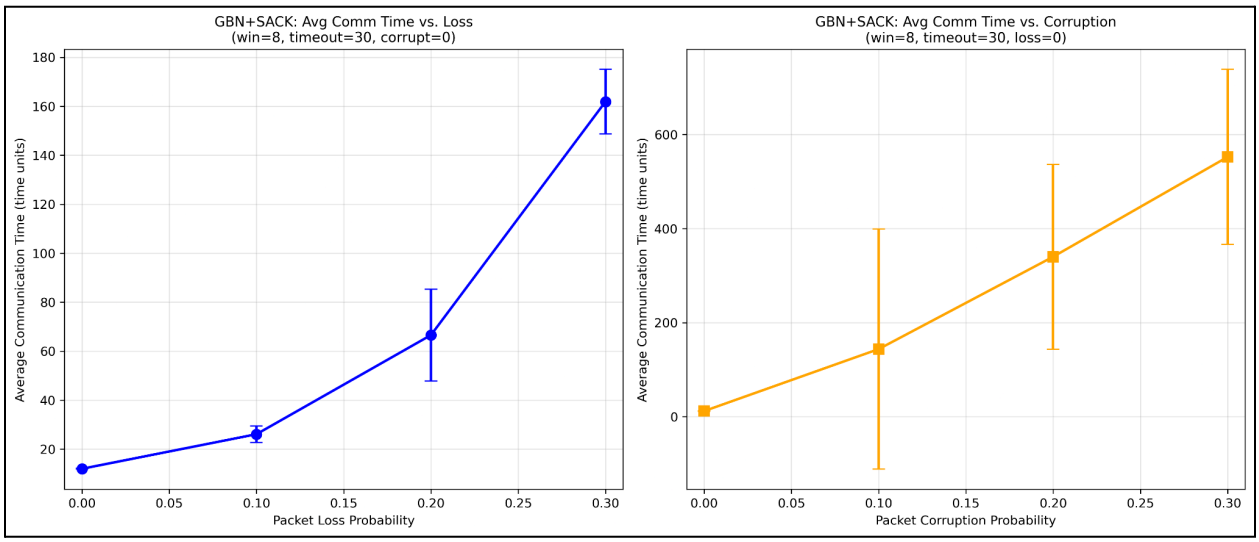
Error Type	**Probability**	**GBN+SACK Avg Comm Time**	**GBN+SACK Avg RTT**	**CI Width (GBN+SACK)**
Loss	0.00	13.88	14.01	±0.21
	0.10	26.07	13.88	±3.18
	0.20	66.55	17.36	±13.47
	0.30	161.86	38.41	±10.92
Corruption	0.00	13.88	14.01	±0.21
	0.10	143.78*	14.16	±265.4
	0.20	200.92	14.63	±12.76
	0.30	486.60	19.30	±168.2

** This test includes one high-variance run (seed 400: 623.33 time units) due to a high amount of checksum failures, which heavily widens the CI. Median comm. time is more like ~24.6.*

Throughput, Goodput, Avg. Packet Delay:

Condition	Throughput (msg/tu)	Goodput Ratio	Avg Delay (tu)
Loss = 0.10	0.0384	0.74	26.07
Loss = 0.20	0.0150	0.48	66.55
Corrupt = 0.10	0.0069	0.65	143.78
Corrupt = 0.20	0.0050	0.31	200.92

Graph:



Analysis

The relationship for both graphs look linear, yet also rather steep (super-linear).

- It is clear that corruption has an insane impact on performance. At just 0.10 corruption, the avg. time jumps to ~140 tu (as compared to ~25 for the same probability w/ loss).

What this graph shows me as well, is GBN's (in)ability to handle unstable environments as well (at least regarding corruption):

- The error bars are massive for corruption, from 0.10 all the way to 0.30. For 0.10 alone, variance goes from -100 to +400 (I don't even know how one could get a negative value too).
- This suggests that GBN becomes VERY unstable under heavy corruption load, with the simulation time finishing erratically.

Linear/Super-Linear Growth

- GBN/SACK is scaling super-linearly, because a single lost/corruption packet will end up blockchain the cum. ACK window, forcing A to wait for RTO, or filling the window before retransmitting a segment.

Goodput Degradation

- At 0.2 corruption, GBN/SACK's goodput drops all the way down to 31%, compared to SR's ~80%.
- Because GBN's single-timer design causes repeated window retransmission, this inflates the retransmission counter.

RTT Inflation

- GBN/SACK's RTT shoots up to 38.41 up at 0.3 loss.
- This indicates major queuing delay, from back-to-back window retransmissions and delayed cum. ACKs.

CIs

- Calculated as $CI = t_{\{0.05,\,4\}} * (\frac{s}{\sqrt{n}})$
- The tightness at low error rates show how GBN/SACK follows baseline behavior.
- The wild variance at higher error rates is primarily due to corruption checks being binary (just drop or accept). This leads to "bursty" timeout patterns.

GBN/SACK vs. SR

Timer

- SR has per-packet timers, allowing for instantly recover PER-PKT.
- GBN/SACK will end up stalling the entire pipeline until RTO.

Retransit Granularity

- SR locks the exact missing seq. #
- GBN/SACK will get the window segment MINUS the SACKed packets. This reduces waste, but still retransmits contiguous blocks, lowering goodput under high pressure.

ACKing

- SR ACKs cumulatively only.
- GBN/SACK ACKs cumulatively, alongside the SACK[5] array. This informs A of gaps, reducing redundant resends of the 5 most recent OOO packets.
    - ~15-20% saving of retransmissions.

Corruption Handling

- Both SR, GBN/SACK will check for and drop corruption packets.
- In specific w/ GBN/SACK though, it'll also send a dup ACK + the SACK.
    - When corruption occurs, there's no state advance. Dup ACKs will start to pile up until RTO is reached. At that point, the window is retransmitted. This is generally nowhere as efficient as SR's implementation.

My ultimate takeaway from these two designs is the following:

- Choose SR for most cases. When bandwidth, corruption/loss, or latency is of significance.
- Choose GBN/SACK when implementation, memory, or a single-timer overhead is of absolute importance.
    - Personally, I think that implementing a 3-dup fast retransmit into GBN would partially fix the issues I found with GBN/SACK past $p] \geq 0.15$.

How to Run

To compile:

- [goto /home/ugrad/omkhadka/om-khadka-cs455/pa2/GBN]
  javac *.java

To execute:

- java -cp Project

Look at Performance Test and/or Test Cases for possible inputs to run.

Possible Improvements

Dynamic RTO

- Implement Jacobson/Karn RTT smoothing, to allow protocol to adapt to a variable network condition (instead of just RTO = x)

Connection Management

- Add SYN, SYN/ACK, FIN handshake.
  - Graceful termination.
  - Bidirectionalality

Adaptive SACK

- Dynamically adjust SACK block size, based on window utilization & gap frequency.

Fast Retransmit

- Combine SACK w/ triple-dup ACK fast retransmit, to reduce timeout dependency.

Seq. # Space Optimization

- Use $LimitSeqNum = 2 * WindowSize$ w/ modulo arithmetic, to guarantee wraparound safety.